

FRONTEND DEVELOPMENT

ECMAScript 6

Guia Técnico e Manual de Funcionalidades

| LET E CONST

Introdução do escopo de bloco e constantes imutáveis. Substitui o uso do 'var' para evitar o içamento (hoisting) e vazamento de escopo.

```
// Escopo de bloco: let e const não saem das chaves
if (true) {
  let userName = "Ana";
  const PI = 3.1415;
}
// console.log(userName); // ReferenceError

// Constantes permitem mutação de propriedades, mas não reatribuição
const user = { id: 1 };
user.name = "Carlos"; // OK
// user = {}; // TypeError
```


| ARROW FUNCTIONS

Sintaxe reduzida e vinculação léxica do 'this'. Perfeito para métodos de array e funções de retorno rápido.

```
// Sintaxe com retorno implícito
```

```
const dobrar = n  $\Rightarrow$  n * 2;
```

```
// Uso em arrays
```

```
const numeros = [1, 2, 3];
```

```
const dobrados = numeros.map(n  $\Rightarrow$  n * 2);
```

```
// 'this' léxico: herda o contexto do pai
```

```
this.valor = 10;
```

```
setInterval(()  $\Rightarrow$  {
```

```
    this.valor++; // Funciona corretamente
```

```
}, 1000);
```


| TEMPLATE LITERALS

Uso de crases (`) para interpolação de variáveis e criação de strings multilinhas de forma legível.

```
const produto = "Smartphone";
const preco = 2500;

// Interpolação com ${}
const mensagem = `O ${produto} custa R$ ${preco}`;

// String Multilinha nativa
const html = `
    ${
        ${produto}
        ${preco}
    }
    `;
```


| DESESTRUTURAÇÃO

Extração simplificada de dados de objetos ou arrays para variáveis distintas com sintaxe limpa.

```
const config = { tema: "dark", lat: -23.5, lng: -46.6 };
```

```
// Extração de objetos
```

```
const { tema, lat, lng } = config;
```

```
// Extração de arrays
```

```
const cores = ["#000", "#fff"];
```

```
const [preto, branco] = cores;
```

```
// Troca de variáveis (Swap)
```

```
let a = 1, b = 2;
```

```
[a, b] = [b, a]; // a=2, b=1
```


| REST E SPREAD

O operador '...' expande iteráveis (Spread) ou coleta múltiplos argumentos em um array (Rest).

```
// Spread: Expandindo elementos
```

```
const arr1 = [1, 2];
```

```
const copia = [...arr1, 3, 4];
```

```
// Spread em Objetos (Fusão)
```

```
const original = { x: 1 };
```

```
const atualizado = { ...original, y: 2 };
```

```
// Rest: Coletando argumentos
```

```
function somar( ...numeros) {
```

```
  return numeros.reduce((acc, n) => acc + n, 0);
```

```
}
```


| MÓDULOS

Sistema nativo de organização de código. Permite exportar e importar funcionalidades entre arquivos diferentes.

```
// arquivo: lib.js  
export const TOKEN = "XYZ123";  
export function validar() { return true; }  
export default class Api {}  
  
// arquivo: app.js  
import Api, { TOKEN, validar } from "./lib.js";  
  
validar();  
console.log(TOKEN);
```


| CLASSES

Abstração sobre protótipos para facilitar a Programação Orientada a Objetos. Suporta herança e construtores.

```
class Pessoa {  
    constructor(nome) { this.nome = nome; }  
    saudar() { return `Olá, eu sou ${this.nome}`; }  
}  
  
class Aluno extends Pessoa {  
    constructor(nome, curso) {  
        super(nome); // Chama o pai  
        this.curso = curso;  
    }  
}  
  
const joao = new Aluno("João", "ADS");
```


| PROMISES

Gerenciamento de operações assíncronas. Substitui os callbacks encadeados por uma estrutura de estados (Pending, Fulfilled, Rejected).

```
const buscarDados = () => {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      const sucesso = true;  
      sucesso ? resolve("OK!") : reject("Erro");  
    }, 1000);  
  });  
};
```

```
buscarDados()  
  .then(res => console.log(res))  
  .catch(err => console.error(err));
```


| ASYNC / AWAIT

Açúcar sintático para Promises. Permite escrever código assíncrono com a legibilidade de um código síncrono (ES2017).

```
async function executar() {  
  try {  
    const resultado = await buscarDados();  
    console.log(resultado);  
  
    const maisDados = await buscarOutraApi();  
    return maisDados;  
  } catch (erro) {  
    console.error("Falhou:", erro);  
  }  
}  
  
executar();
```


| MAP E SET

Novas coleções: 'Map' para mapeamento chave-valor (permite chaves de qualquer tipo) e 'Set' para listas de valores únicos.

```
// SET: Garante valores únicos  
const numeros = [1, 1, 2, 3, 3];  
const unicos = [...new Set(numeros)]; // [1, 2, 3]  
  
// MAP: Dicionário flexível  
const mapa = new Map();  
mapa.set("id", 10);  
mapa.set({ obj: 1 }, "valor");  
  
console.log(mapa.get("id")); // 10
```


Conclusão

O ES6 mudou fundamentalmente a maneira como escrevemos JavaScript, trazendo robustez, legibilidade e ferramentas modernas para o ecossistema Web.

Fim da Apresentação